

Linux 的发展

→ 在一般操作系统的功能上增加了网络功能
Network Operating System
Linux 是对网络支持得最好的 OS 之一

Linux 是一个网络操作系统 (NOS), 具体包括:

- 1) 实现网络中各计算机之间的通信和资源共享
- 2) 提供多种网络服务软件
- 3) 提供网络用户的应用程序接口

其与其他商业化的网络操作系统不同,完全免费,且与UNIX完全兼容,性价比高

Linux 最早是 Linus Torvalds 于 1991 年在芬兰赫尔辛基大学原创开发的, 并在 GNU 的 GPL 原则下发行

Linux 的版本

Linux 的版本号分为 { 内核 (Kernel) 版本 2 个部分
 发行 (Distribution) 版本

其中,内核版本的序号由 major. minor. patchlevel 3个部分组成

主版本号 次版本号 修订版本号

国外主要发行版本:

- Real Hat Linux
- SUSE Linux
- Debian Linux
- Ubuntu Linux
- CentOS Linux

国内主要发行版本:

- 红旗 Linux
- 中标普华 Linux
- 共创桌面 Linux
- 冲浪 Linux
- 蓝点 Linux

Linux 的特性

- ① 多任务 多用户
- ② 支持多种文件系统
- ③ 采用虚拟内存管理技术
- ④ 良好的可移植性
- ⑤ 设备独立性
- ⑥ 丰富的网络功能
- ⑦ 提供全部源代码

可同时运行多个程序或执行多个任务，
通过进程调度来管理这些任务的执行。

Linux 的功能

- ① 稳定的核心
- ② 丰富的应用软件
- ③ X Window 系统 (Linux 图形界面的底层显示系统)
- ④ 多重启动
- ⑤ 网络功能
- ⑥ 软件开发工具

多个用户可以同时登录并使用同一台主机或服务器,系统为每个用户提供独立的账号、密码、文件和权限管理,互不干扰

Linux 的常用软件

- 1) WWW 服务器: Apache、Caudium、CERN HTTPD Server、Netscape Fast Track Server、Zeus Server
万维网
- 2) FTP 服务器: Wu-ftp、proftpd、vsftpd
文件传输协议
- 3) FTP 客户端: gFTP、lftp
- 4) Mail 服务器: Qmail、sendmail、XMail
- 5) E-Mail 客户端: pine、Kmail
- 6) 浏览器: Netscape、Mozilla、Phoenix、lynx
- 7) 虚拟终端: rxvt、xterm、gnome-terminal、m/term
- 8) 聊天工具: gaim、Liq
- 9) 网络共享服务: samba
- 10) 远程登录: telnet、ssh1、ssh2
安全的远程登录协议

X Window

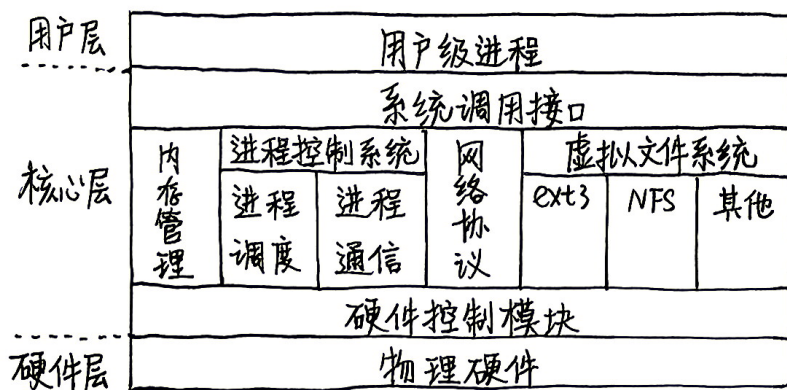
该系统是 Linux 上的 标准图形界面，能有效支持多种应用环境。

可在 X Window 基础上实现不同的桌面系统，如：KDE、GNOME 桌面系统

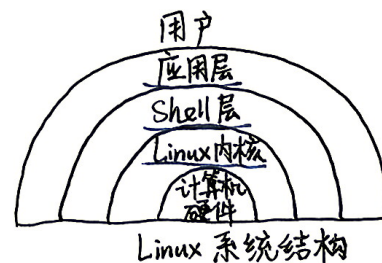
多媒体软件

- 1) GNOME CD Player 是一款基于 X Window 的 CD 播放软件。
- 2) Realplayer: RM 格式的影音文件是网上广为流传的电影格式，以其极高的压缩率而闻名，Linux 下亦可看 RM 格式的文件，这就是 Realplayer for linux。
- 3) XMMS (X Multi Media System) 是 Linux 下的 MP3 播放器
- 4) Mplayer 是个 VCD 播放软件
- 5) KSnapshot 是个基于 KDE 桌面环境的抓图软件
- 6) GIMP 是个免费的、分布式的图片润饰、图像制作和处理软件

Linux 的内核体系结构



Linux 系统核心框图



Linux 系统大致可分为 3 层：

靠近硬件的底层是 内核，即 Linux 操作系统常驻内存的部分；

中间层是内核之外的 Shell 层，即操作系统的系统程序部分；

最高层是 应用层，即用户程序部分，包括文本处理程序、语言编译程序及游戏程序。

Linux 主要应用于 服务器、嵌入式系统、桌面

Linux 系统可以在 2 种环境下操作:

- ① 在 X Window 的桌面环境下操作, 与 Microsoft Windows 环境下的操作十分相似
- ② 在控制台的字符屏幕下操作, 通过键盘输入命令来实现对系统的操作

Linux 常用命令

Shell 是一种命令解释程序 (Command-Language Interpreter), 负责用户与操作系统的沟通

在 Linux 的终端操作方式 (即命令行方式) 下, 会出现一个 Shell 提示窗口, 用户可以在这个

窗口里输入命令以操作系统

[root@linux root] #

主机名

当前目录 (/root)

[root@localhost ~] #

主机名

当前目录 (~ 是用户主目录的简写,

这里恰好 root 用户的主目录就是 /root)

管理员

普通用户的主目录是 /home/用户名

(一) 进入系统

login:

新用户应联系系统管理员取得合法登录名

Enter

password:

输入的口令字符串不显示以利于保密

Enter

系统对登录名与口令验证无误后, 则在屏幕显示若干行信息,

最后一行出现 \$- (\$ 是一般用户的 Shell 提示符, 其后是闪烁的光标条)

root 用户的是 #

退出系统

在 \$ 提示符出现后, 输入 logout

Enter / Ctrl + D

login:

表示已退出, 重新登录方可再次进入系统

(二) 系统询问命令

① date

询问当前日期和时间

② who

询问当前用户, 可列出当前每一个处在系统中的用户的登录名、终端名、登录进入时间

③ pwd → print working directory (打印当前工作目录).

显示当前目录路径

(三) 文件操作命令

Linux 命令行的一般格式为：命令名 [选择项] [参数]

Linux 的文件通常分为4类：普通文件、目录文件、设备文件、链接文件。

为对文件进行保护，Linux 系统提供了文件存取控制方式，把所有用户划分为3种身份：文件主 (User)、同组用户 (group)、其他用户 (other)，每种用户对一个文件可拥有读 (r)、写 (w)、执行 (x) 的权力。

read write execute

Linux 文件权限的字符表示：

通常由1位文件类型位和9位权限位组成

-	普通文件
d	目录文件
c	字符设备文件
b	块设备文件
l	链接文件

分别依次表示 user、group、other 用户的 r、w、x 权限

rwxr-xrw-

相应用户不具有相应权限

文件显示命令

cat (concatenate) 用于显示文件内容，也可以把多个文件内容连接起来输出

cat 文件名 显示单个文件内容

cat 文件名1 文件名2 合并多个文件内容

cat 文件名1 文件名2 ⑤ 文件名3 合并文件1、2的内容，然后写入文件3
输出重定向

cat [-u] [-s] [-v] [-t] [-e] 文件名 ...

↓	↓	↓
不使用缓冲，立即输出	显示不可见字符	显示换行结束符 (\$)

压缩连续空行 显示制表符 (\\t)
成1行

cat 或 cat - 从标准输入读取 (读键盘输入)，并输出内容到屏幕

cat 命令用来连接并显示文件，它顺序阅读每个文件，并将它们在标准输出上输出。

文件内容比较命令

1) comm 用于比较2个已经排好序的文件

comm [-123] 文件名1 文件名2

默认情况下, comm 文件名1 文件名2 会输出3列 (差异与共同内容)

第1列是只在文件1中出现的行, 第2列是只在文件2中出现的行, 第3列是2个文件都有的相同行

如果前面有 "-", 就表示不显示该列该内容

例如 comm -12 文件名1 文件名2, 不显示第1、2列, 只显示二者共有内容

2) diff 用于比较2个文本文件的不同之处

diff 文件名1 文件名2

它会逐行^{旧'}比较, 指出^{新'}哪些行被增加/删除/修改了

如果二者完全一样, 通常没有输出

diff [-br] [-c] [-n] [-e] [-f] [-h] 文件名1 文件名2

-b: 忽略空格数差异
-c: 以上下文格式显示差异
-n: 指定显示差异前后n行上下文
-e: 反向顺序输出差异补丁
-f: 快速简化对比, 不检查复杂换行格式
-h: 递归对比整个文件夹里所有文件

输出ed编辑器格式的补丁命令

3) cmp 用于比较2个文件(是否)相同

cmp 文件名1 文件名2

它会按字节比较, 如有不同, 会显示第一次不同的位置

如果二者完全一样, 通常没有输出

cmp [-l] [-s] 文件名1 文件名2

↓
列出所有不同字节的位置

↓
静默模式, 不显示结果, 只通过返回值

{ 0 相同
1 不同
2 文件找不到/报错

来判断是否相同

文件的复制、移动和删除命令

1) cp 用于复制文件或目录

cp  源文件 目标文件

cp  源文件1 源文件2 目录 (cp a.txt b.txt /home/newland/)

★ 若复制多个文件, 最后一个参数必须是目录

★ 复制目录时要加 -r 或 -R, 把目录1 及其中所有内容复制到目录2

cp -r 目录1 目录2

有: 目录

无: 目录或文件



[-fip]

-f: 强制复制, 覆盖目标文件

-p: 保留原文件属性 (如时间、权限等)

-i: 覆盖前询问

2) mv 用于移动或重命名文件和目录

mv  源文件 目标文件 (重命名)

mv  源文件 目录 (mv file1 /home/newland/)

★ 移动多个文件到目录

mv 源文件1 源文件2 目录

[-fi]

-f: 强制复制或覆盖

-i: 覆盖前询问

3) rm 用于删除文件或目录

rm  文件名

★ 同时删除多个

rm 文件名1 文件名2 文件名3 ...

[-firR]

-f: 强制删除, 不提示

-r 或 -R: ★ 删除目录

rm -r 目录

-R

-i: 删除前逐个询问

确定文件类型的 `file` 命令

当用户拥有很多文件时,常以自己的登录名为根节点,为自己建立一棵文件子树。

其中,一些节点可能是目录文件和另一些可能是信息文件。若用户忘记,则可用 `file` 命令判定。

该命令带有一个参数表,用于给出想了解的某类型的文件名清单。执行结果是在屏幕上显示出各文件的类型

`file` 文件名

☆ 若一次确定多个

`file` 文件名1 文件名2 文件名3 ...

例. `file a.txt p.jpg program`

→ `a.txt`: ASCII text (文本文件)
`p.jpg`: JPEG image data (JPEG 图片文件)
`program`: ELF 64-bit executable (Linux 可执行程序)

`file /home/newland ; ?`

→ `/home/newland`: directory (目录文件)

`file link.txt`

`link.txt` → `a.txt` 打开前者,实则访问后者内容

→ `link.txt`: symbolic link to `a.txt` (链接文件, 指向 `a.txt`)

`file /dev/sda`

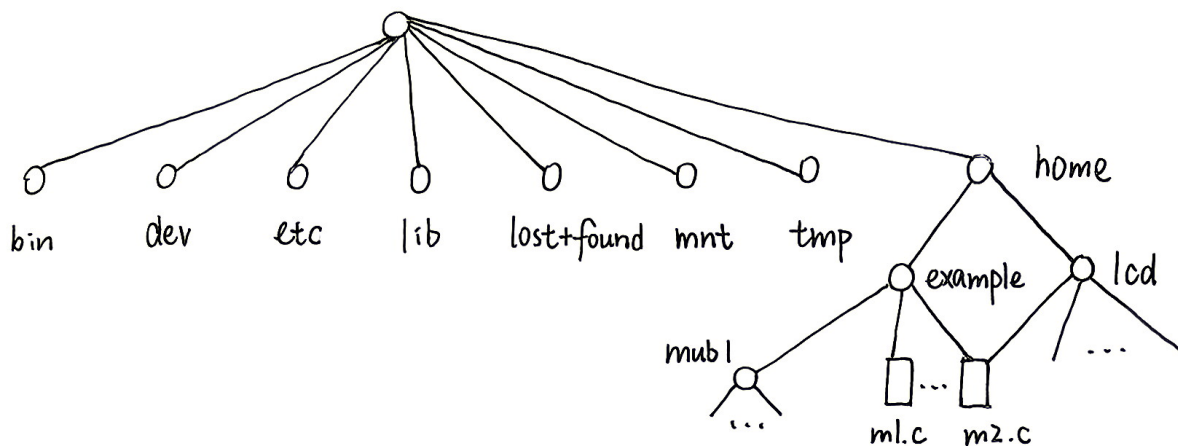
→ `/dev/sda`: block special (块设备文件)

(四) 目录操作命令

目录结构

Linux 文件系统采用带链接的树形结构,即只有一个根目录(通常用"/"表示),其中含有下级子目录或文件的信息,子目录中又可含有下级的子目录或文件的信息...

一层层延伸下去形成一棵倒置的树
根在上面



/bin 存放系统必备的二进制可执行文件(比如 ls, cp, mv, rm, cat, mkdir, sh 这些命令)

/boot 存放系统启动时要用到的文件

电脑开机后,首先由 BIOS/UEFI 完成硬件自检,随后加载硬盘上的 boot loader (如 GRUB).

boot loader 读取 /boot 目录下的 Linux 内核文件与初始化内存盘并将其加载入内存.

内核随后接管系统,通过初始化内存盘识别硬件并挂载根文件系统最终启动用户空间服务

/dev 存放外部设备对应的“文件接口”(并不是放外部设备的驱动程序)

应用程序无需关心底层硬件的差异,只需读写 /dev 下的设备文件就可与之通信

例如给串口发数据: `echo "20050816" > /dev/ttyUSB0`

/etc 存放系统管理时要用到的各种配置文件和子目录

例如网络配置文件、文件系统、X 系统配置文件、设备配置信息、设置用户信息等

/sbin 存放系统管理员的系统管理程序 /home 是存放各个普通用户主目录的上级目录

/home 若建立一个用户“yang”,那么 /home 目录下就有一个对应的 /home/yang 路径以存放用户的主目录, /home 是普通用户的家目录 (/root 是超级管理员的主目录)

/lib 存放系统动态链接共享库,几乎所有应用程序都用到

/mnt 一般为空,用于临时挂载文件系统

/proc 是虚拟文件系统,其内容并不实际保存于硬盘而是系统运行时动态生成

可在该目录下获取系统信息,这些信息在内存中,由系统自己产生

/tmp 存放不同程序执行时产生的临时文件

/usr 存放系统中大部分应用程序与相关资源,是占用硬盘空间最大的目录

文件和目录的路径名可用2种方式表示:

绝对路径名 (亦称: 全路径名) 和 相对路径名

1) 绝对路径

一定从根目录"/"开始

不管当前在哪个目录, 这个路径都能准确地表示同一位置

例如 `cat /etc/passwd`

不论当前在 `/home/yang` 还是 `/tmp`, 都会查看同一文件: `/etc/passwd`

2) 相对路径

不从根目录开始, 以当前所在文件夹为起点, 往下找

例如

当前在 `/home/yang`, 当前目录下有一个文件 `a.txt`

那么可以直接写 `cd a.txt` (完整对应的绝对路径是 `/home/yang/a.txt`)

"." 表示当前目录

假如当前在 `/home/yang`, 那么"."表示 `/home/yang`

"./start.sh" 表示运行当前目录下的 `start.sh` 文件

".." 表示上一级目录

假如当前在 `/home/yang/success`, 那么".."表示 `/home/yang`

如果执行 "`cd ..`" 就表示返回上一级目录

"`cd ../download`" 就表示先返回上一级目录, 再进入 `download` 目录
(即 `/home/yang/download`)

目录的创建和删除

1) `mkdir` 用于创建目录

创建单个: `mkdir 目录名1`

创建多个: `mkdir 目录名1 目录名2 目录名3 ...`

如目录名中有空格, 应用引号括起来:
`mkdir "my documents"`

`[-ep] [-m mode]`

`-p`: 递归创建多级目录,
父目录不存在时自动创建

`-m mode`: 创建时直接设置目录权限

例如 `mkdir -m 755 dir`

创建权限为 `rw-r-xr-x` 的目录

例如 `mkdir -p a/b/c`

一次性创建 `a`、`a/b`、`a/b/c` 三层目录

例如 `mkdir -p src/include lib bin`

一次性创建 `src`、`src/include`、`lib`、`bin` 4 个目录

2) `rmdir` 用于删除目录, 但其不能删除安装文件系统的根目录 (例如 `/bin`、`/etc`、`/usr`、`/home`)

`rmdir 目录名`

★ 只能删除空目录, 一旦目录里面还有文件或子目录, 就会删除失败, 报错“非空”

以下 3 种写法, 都指向同一个目录, 结果相同:

`rmdir /home/yang/test`

`rmdir /home/yang/test/`

`rmdir test` # 当前目录是 `/home/yang` 时.

`[-r] [-p] [-s]`

`-r`: 可以删除有内容的目录

例如 `rmdir -r /a/b/c` 先把 `/a/b/c` 中的文件、子目录删除, 再把 `/a/b/c` 目录本身删除, 父目录 `/a` 和 `/a/b` 完全不受影响

`-p`: 从子往父删除, 只要中间某一级目录不是空的, 就会停止

例如 `rmdir -p /a/b/c` 先删空的 `c`, 再删空的 `b`, 再删空的 `a`

`-s`: 静默模式, 删除时不输出任何提示信息

目录的显示和改变工作目录

1) `ls` 用于显示当前目录下有哪些文件和目录

`ls`

a.txt test hello.c

`ls -l`

只显示文件名而不显示细节

显示详细信息 (权限、所有者、大小、修改时间、文件名等)

`ls -a`

显示以点开头的隐藏文件

drwxr-xr-- yang yang 4096 May 10 16:18 a.txt

.bashrc .profile .config

2) `cd` 用于切换工作目录

`cd` 目录名

例如 `cd /home/yang` 表示进入 `/home/yang` 目录,

若当前目录下有一个 `test`, 则 `cd test` 表示进入 `/home/yang/test`

此时 `cd ..` 就会回到 `/home/yang`

`cd /` 表示进入根目录 (根目录是整个 Linux 文件系统的顶层)

☆ `cd` 或 `cd ~` 都会回到用户的家目录 (`/home/yang`)

用户名

3) `pwd` 用于显示当前目录

假设当前在 `/home/yang` 目录下

`pwd`

输出: `/home/yang`

然后进入 `/boot/grub` 目录

`cd /boot/grub` (绝对路径)

`pwd`

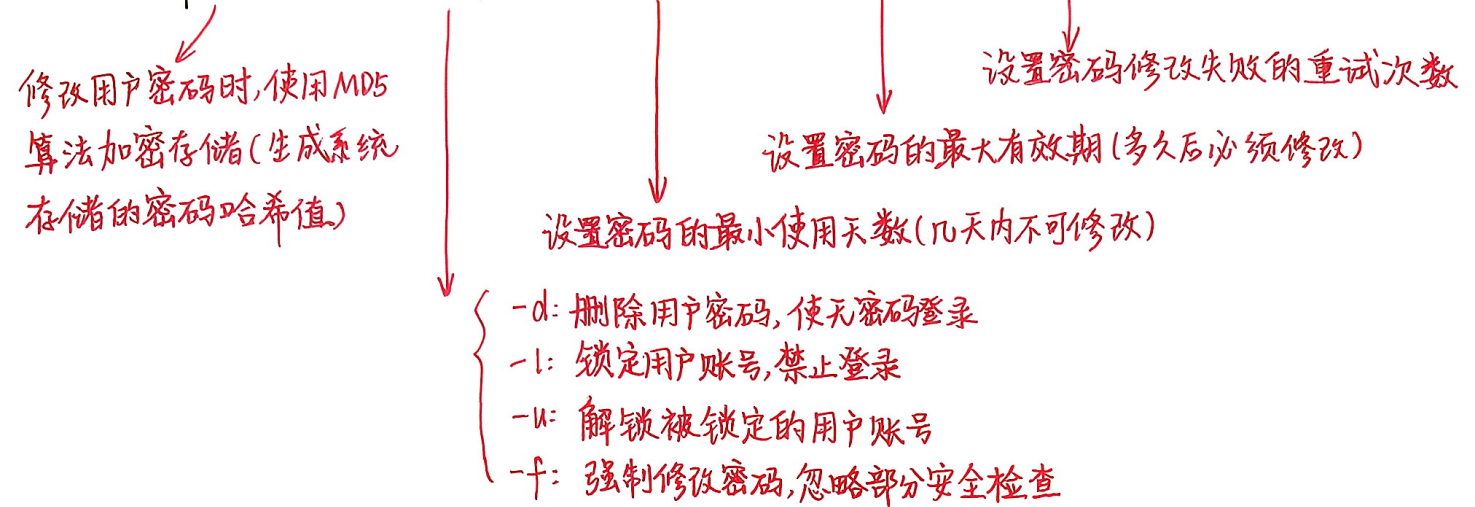
输出: `/boot/grub`

(五) 口令、权限命令

修改口令

passwd 的功能是改变登录或 modem (调制解调器) 拨号 shell 口令

`passwd [-m] [-d|uf] [-n minimum] [-x expiration] [-r retries] [用户名]`



`passwd -s [-a] [用户名]`

与 `-s` 一起用, 查看所有用户的密码状态

查看用户的密码状态 (是否锁定、有效期等)

基础用法: 直接输入 `passwd` 命令, 按提示输入旧密码后再输入新密码 (2 次) 即可完成修改

管理员用法: root 用户可以直接修改任意用户的密码且无需知道旧密码, 例如 `password liu`

改变存取权限

chmod 的功能是改变文件或目录的存取权限

① 符号方式

chmod [-R] [who] [+|-|=] [mode ...] 文件名...

增加权限

→ 去掉权限

直接设置为指定权限, 覆盖原来的

给谁改权限 {
u: 所有者
g: 同组
o: 其他
a: 所有用户, 默认

递归修改权限, 修改目录本身与目录里所有文件/子目录的权限

例如

`chmod u+x test.sh` # 给所有者加执行权限

`chmod go-rw test.txt` # 给同组和其他用户去掉读、写权限

`chmod a=r test.txt` # 给所有用户设置只读权限

② 绝对方式 (可使用8进制数来设置文件、目录的权限)

chmod [-R] mode 文件名/目录名 ... 是目录的要注意!

1个3位8进制数, 每位数字对应 u/g/o 的权限 r=4, w=2, x=1

$$\begin{cases} rwx = 4+2+1 = 7 \\ rw- = 4+2 = 6 \\ r-x = 4+1 = 5 \\ r-- = 4 \end{cases}$$

改变文件和目录所有权

chown [-cfvR] 用户名 文件和目录列表

用于重新指定所有权的用空格分隔的文件和目录列表

可能是用户号或文件 /etc/passwd 中出现的用户名

- C: 仅在文件权限确实改变时才进行详细说明
- f: 不打印权限不能改变的文件的错误信息 (没权限改时不报错)
- v: 详细说明权限的变化
- R: 递归改变目录及其内容的权限

目录本身及目录里所有文件、子目录的所有者

(六) 文件压缩与联机帮助命令

gzip 压缩/解压缩文件

gzip [-cdlr tv] 要压缩/解压缩的文件名

-c: 保留原文件, 将结果输出到标准输出 (压缩)

即, 压缩结果会先输出到屏幕 (标准输出), 可以把这个输出 "重定向" 到一个新文件里, 这样原文件不会被删除

例如 `gzip -c test.txt > test.txt.gz`

解压一定要带它

-d: 把 .gz 格式的压缩文件恢复成原始文件 (解压)

例如 `gzip -d test.txt.gz` # -dc 则解压后不会删除 .gz 文件

`gzip -d *.gz` # 批量解压当前目录下所有 .gz 文件

-l: 对每个压缩文件显示压缩后大小、原文件大小、压缩率、原文件名等

↓ compressed ↓ uncompressed ↓ ratio

例如 `gzip -l test.txt.gz`

~~注意, 压缩率是 n% 说明被压缩掉了 n% 的体积~~

整个目录打包

-r: 递归压缩/解压目录中的所有文件

注意与 tar 的区别!

`gzip -r logs/` # 递归压缩 logs 目录下的所有文件 (每个文件单独生成 .gz)

`gzip -dr logs/` # 递归解压 logs 目录下的所有 .gz 文件

-t: 检查 .gz 文件是否损坏, 能否正常解压, 避免解压途中发现文件损坏

-v: 对每个压缩和解压的文件, 显示文件名和压缩比

例如 `gzip -v test.txt` # 带过程信息压缩

`gzip -dv test.txt.gz` # 带过程信息解压

→ 生成的 .tar 是一个文件!

tar 打包文件 / 目录

可对文件和目录进行打包, 既可对某一特定文件进行打包 (一般用作备份文件), 也可在包中改变文件, 或向包中加入新文件

tar [主选项 + 辅选项] 文件或目录

主选项:

-c: 创建新的档案文件, 如果用户想备份一个目录或是一些文件, 就要选择该选项

tar cf 备份包.tar 目录/

会把“目录/”里的所有文件打包成备份包.tar

-x: 从档案文件中释放文件 (把备份包.tar 中的文件解压到当前目录)

tar xf 备份包.tar

释放

-r: 把要存档的文件追加到档案文件的末尾, 可将忘记的目录或文件追加至备份文件中

tar -rf 备份包.tar 新增的文件名

-t: 列出档案文件的内容, 查看已备份哪些文件

tar -tf 备份包.tar

-u: 更新文件, 用新增文件替换^{同名的}原备份文件, 若在备份文件中找不到要更新的文件则把它追加到备份文件的最后

tar -uf 备份包.tar 已修改的文件名

-c 打包

-x 解压

-z 压缩

①

“管道 (pipeline)” : 把前一个命令的输出结果直接传给后一个命令作输入
不用存文件, 直接接力处理

辅助选项:

-f: 指定档案文件名/设备, 告诉 tar 要打包的东西叫什么名字

tar -cf 我的包.tar 文件名1 文件名2 文件名3 ...

-k: 解包时如遇同名文件, 保留已存在的而不覆盖

tar -xkf 备份包.tar

-m: 解包 (还原文件) 时, 把所有文件的修改时间设为当前而非打包的时间

tar -xmf 备份包.tar

-M: 创建多卷档案 (分卷打包), 把大包拆成多个

tar -cMf 分卷包.tar 目录/

tar 开始打包, 先写数据到分卷包.tar 中, 当文件大小达到限制则暂停打包, 提示“换盘”, 插入新的存储介质后会继续生成“分卷包.tar.1 ...”分卷文件。

-v: 显示详细处理过程, 实时显示正在打包/解包的文件而不静默

tar -cvf 备份包.tar 目录/

-Z: 用 gzip 压缩/解压缩, 打包后自动用 gzip 压缩, 生成 .tar.gz 文件, 解压时自动解 .tar.gz

tar -czf 备份包.tar.gz

tar -xzf 备份包.tar.gz

现在的分卷打包方法:

方法1: 先打包, 再 split 分卷

tar -czf 大包.tar.gz 目录/

split -b 4G 大包.tar.gz 分卷包-

换 4GB 大小分卷

生成 分卷包-aa、分卷包-ab、...

要恢复时, 先拼起来再解压

cat 分卷包-* > 大包.tar.gz

tar -xzf 大包.tar.gz

方法2: 结合“管道”, 打包+分卷一步到位

tar -czf - 目录/ | split -b 4G - 分卷包-

(七) 联机帮助

man 用于显示帮助手册，通常只需在命令 **man** 后输入想要获取的命令的名称 (例如 **ls**)，**man** 就会列出一份完整的说明，其内容包括命令语法、各选项意义及相关命令等

man [选项] 命令名称

-s: 根据章节显示 (一个命令名称可能会有很多类别)

man 命令的章节常用选项:

章节	说明
1	一般使用者的命令
2	系统调用的命令
3	C 语言函数库的命令
4	有关驱动设备和系统设备的解释
5	配置文件的解释
6	游戏程序的命令
7	其他软件或程序的命令和有关系统维护的命令

章节的作用: 告诉 **man** 想看哪一类说明

man passwd 查看 **passwd** 命令，而 **man 5 passwd** 查看 **passwd** 配置文件的说明

-f: 只显示出命令的功能而不显示其中详细的说明文件

→ “不显示手册，只显示将源代码格式化和显示的文件的所在位置”

-w: 不显示手册内容，只显示手册文件的路径，即:

查看该命令的手册文档的实际位置，跳过“解压→格式化→显示”的步骤

-a: 显示所有同名的手册页

-E: 在手册每行的末尾显示 \$ 符号，以便复制多行内容时区分行边界
(防止复制时丢掉换行符)

Shell 程序设计

Shell 是 Linux 系统中的一个重要层次,是用户与系统交互作用的界面

终端 (Terminal) 是黑框窗口,负责显示文字、接收键盘输入,

Shell 是运行在终端里的程序,负责处理用户输入的程序,与内核交互

Shell 除了作为命令解释程序以外,还是一种高级程序设计语言。
利用 Shell 程序设计语言可以编写出功能很强但代码简单的程序。

Shell 的特点和主要版本

在 Linux 系统中通常提供多种不同的 Shell,常用的有

Bourne shell	(sh)
C-shell	(csh)
Korn shell	(ksh)
Bourne Again shell	(bash)

* bash 是 Linux 中一个默认的 shell

Shell 亦可存放在文件上。例如:由 3 条简单命令组成的 shell 程序(文件名 ex)。

date # 显示当前系统的日期和时间

pwd # 显示当前所在文件表路径 (Print Working Directory)

cd .. # 切换到上一级目录 (.. 代表当前目录的父目录)

Shell 过程的建立和执行

内容是给 Shell 执行的命令

称作 Shell 文件或 Shell 脚本 (Shell script), 建立 Shell 过程的步骤与

建立普通文本文件相同,利用编辑器 (如 vi 或 gedit) 进行程序录入和编辑加工

内容是给人看的文字

执行 Shell 过程的方式主要有以下 3 种:

1) 输入定向到 Shell 过程

输入重定向

例如: sh ex # 把 ex 文件里的内容当作输入,交由 sh (Shell 程序) 执行

2) 以过程名作参数 (直接调用 Shell 程序,把脚本文件当作参数传给它)

例如: sh ex # 让 sh 读取并执行 ex 这个文件里所有的命令

3) 将 Shell 过程改为可执行文件 (给脚本文件加上 "可执行权限", 直接当程序运行)

例如: chmod +x ex
./ex

insmod 加载 .ko (内核模块/驱动) 文件到 linux 内核

sudo 临时切换成 root (管理员) 身份

grep 在文件里搜索关键字

Shell 变量

依据生命周期和作用范围, 分成 { 环境变量 临时变量 } 2 种:

- 环境变量是永久性变量, 其值不会随 Shell 过程执行过程结束而消失
- 临时变量是在 Shell 程序内部定义的, 其使用范围仅限定义它的程序, 出了该程序就不能再用了. 并且, 当程序执行完毕, 它的值也就不存在了.

在 Shell 过程中, 临时变量又分为 { 用户定义的变量 位置参数 } 2 种

- 用户定义变量是用户自己在脚本里写的变量, 例如: `dir = /home/yang/project`
- 位置参数是脚本运行时传给它的参数, 例如: `./test.sh a b c` 中的 `a b c`

用户定义的变量

定义变量并赋值的一般形式是: 变量名 = 字符串

例.

`dir = /home/yang/project` # 给变量 `dir` 赋值

`echo $dir` # 用 `echo` 取出变量的值并用 `echo` 打印出来, 输出 `/home/yang/project`

`echo dir` # 把 "dir" 当普通字符打印, 输出 `dir`

输入/输出命令

键盘输入

1) `read` 用于从标准输入读取数据, 然后赋给指定变量
`read 变量1 [变量2 ...]`

例如, `echo "请输入 2 个数字:"`

(脚本) `read a b`

`echo "第 1 个是 $a, 第 2 个是 $b"`

2) `echo` 用于将其后的参数在标准输出上输出, 各参数隔以空格, 以换行符终止

例如, `echo "今天是" $(date +%Y-%m-%d)`, 当前目录是 "`$PWD`"

↓
`/home/user`

Shell 中的特殊字符

① 通配符 用于模式匹配, 如文件名匹配、路径名搜索、字符串查找等

例如:

* 匹配任意长度的任意字符 (包括空字符) `ls *.txt` 列出所有 .txt 文件

? 匹配单个任意字符

`ls file?.txt` 匹配 `file1.txt`, `fileA.txt`, 但不匹配 `file0.txt`

[] 匹配括号里任一字符

`ls file[123].txt` 匹配 `file1.txt`, `file2.txt`, `file3.txt`

② 引号

单引号括起来的字符均作为普通字符出现

双引号括起来的字符, 除 $\$$ 、 $\backslash$ 、 $\`$ 仍保留其特殊功能外, 其它字符常作普通字符

1) 取出变量的值: $\$$ 变量名

`name = "小杨"`
`echo "Hello, $name!"` → `Hello, 小杨!`

`echo "当前时间: $(date +%H:%M:%S)"`

→ 当前时间: 22:36:10

2) 执行命令并取出结果 $\$(命令)$ 或 ``命令``

`echo "当前目录是: `pwd`"`

$\Leftrightarrow \$ (pwd)$

→ 当前目录是 /home/yang

被解析成
命令执行结果

若 `path = "/home/yang's 文件夹"`

Shell 中, 空格是命令参数的分隔符

`ls $path` → $\#$ `ls` 找不到 /home/yang's 和文件夹

`ls "$path"` → 正常 并用双引号, 整个路径被当成一个参数

单、双引号的对比

避免命令执行 `name = "小杨"`

出错

`echo "$name 你好! 请支付 \$100"` → 小杨你好! 请支付 \$100

`echo '$name 你好! 请支付 $100'` → \$name 你好! 请支付 \$100

3) 转义符 用于将它后面跟着的特殊字符变普通, 或表示特殊字符

`echo "价格是 \$100"` → 价格是 \$100 $\backslash n$ 换行 $\backslash t$ 制表符

`echo "他说: \"你好!\""` → 他说: "你好!"

`echo "第1行 \n 第2行"` → 第1行
第2行

③ 输入输出重定向

执行一个Shell命令时通常会自动打开3个标准文件,即:

1) 标准输入文件 (stdin), 通常对应终端的键盘

2) 标准输出文件 (stdout)

3) 标准出错输出文件 (stderr)

对应终端的屏幕

Shell中, 这3个文件均可通过
重定向符进行重新定向

输入重定向符 "<" 用于将命令或可执行程序的标准输入重定向到指定文件

例如, cat命令默认等键盘输入内容再打印

cat # 等键盘输入, Ctrl+D结束

cat < data.txt # 让cat从data.txt文件中读内容以代替键盘输入

又例如, 写了一个脚本, 需输入用户名、密码, 若想让其从文件中自动读取。

./start.sh < login-info.txt

输出重定向符 ">" 用于命令或可执行程序的标准输出重定向到指定文件

这样, 输出不会在屏幕上显示, 而是写入指定文件中

例如, who命令本来是在屏幕上显示当前登录用户

who # user1 pts/0 2026-05-13 16:50 (192.168.0.100)

who > output.txt # 屏幕上无显示, 但当前目录下会生成一个保存着 who命令
输出结果的 output.txt 文件

又例如, 写了一个脚本 start.sh

./start.sh # 正常输出到屏幕 ./start.sh > output.txt # 重定向到文件

写了一个 test.c → 编译成可执行文件(gcc test.c -o test) → 运行(./test) # 输出到屏幕

./test > output.txt # 写入文件

./test.py > output.txt 一样生效

> 是覆盖写入: 若文件已存在, 会清空原有内容再写入新内容 echo "123" > test.txt 文件中只有 123

>> 是追加写入: 若文件已存在, 会将新内容加到文件末尾 echo "456" >> test.txt 文件中是 123\n456

④ 注释、管道线和后台命令

1) 注释

Shell 程序中以 # 开头的正文行表示注释

2) 管道线

是由竖杠隔开的若干个命令组成的序列

前一个命令的输出, 正是下一个命令的输入

- 管道左边的标准输出(本来要打印到屏幕), 直接传给右边的命令, 作其标准输入(不用键盘输入)

- 每个命令都是独立的进程, 数据在它们中间“流”过去, 无需手动保存文件再读取

例如, `ls -l $HOME` ① `wc -l` → ② 统计输入内容的行数, 即文件数量

① 列出用户主目录下的所有文件, 输出一长串文件列表

多段管道
(1个管道线中
可以包括多条
命令)

`ls | grep m?.c | wc -l`
↓
把列表传给 `wc -l` 命令

管道线中有类命令 都叫“过滤器”
`grep, wc, sort, tail ...`

① 列出当前目录的所有文件名

② 统计筛选出的文件数量

③ 从列表中选出以 `m` 开头且后跟1个字符且 `.c` 结尾的文件

首先读取输入, 再对输入以
某种方式变换(“过滤”),
最后输出处理结果

3) 后台命令

前台方式: 通常, 在主提示符后输入的命令都立即得到执行。

用户输入数据 → 系统进行处理 → 输出运行结果

执行过程中, 用户与系统发生交互

但是有些程序的运行要花费较长时间, 例如调用 C 编译 (Compiler) 对 C 程序进行编译。若想在编译同时做其他事, 则需输入命令

`gcc (注)
cc test.c &`

在一条命令的最后输入“&”符号, Shell 在后台启动该程序而不占终端。

马上显示提示符 — 提示用户输入新命令

`$HOME` 与 `/home` 不一样:

`$HOME` 是该用户自己的家目录(个人文件夹), 是环境变量

普通用户: `/home/用户名`

root 用户: `/root`

`/home` 是存放所有用户家目录的总目录(系统目录)